

GPU decisions. Fixed buffer shapes. Write control flow with `torch.cond`, `torch.while_loop`, or `torch.switch`; PyTorch captures it as CUDA graph conditional nodes.

Support matrix

Control-flow ops and the CUDA graph node each lowers to

	cond IF / ELSE	while_loop WHILE	switch SWITCH
PyTorch 2.14	✓	✓	✓
CUDA graph support	✓	✓	🚧
In-place input mutation*	✓	✓	✓
Minimum CUDA	12.4	12.4	12.8
Inductor backend	✓	✓	✓

✓ supported in PyTorch 2.14 🚧 open PR, or not yet supported

* Inputs mutated in place under `torch.no_grad()` or `torch.inference_mode()`.
`switch` is `torch._higher_order_ops.switch`; CUDA graph capture needs #189461.
Inductor runs the control flow on the host; it emits no conditional nodes yet.
Checked on PyTorch main (91a0eda); every feature above is in 2.14.

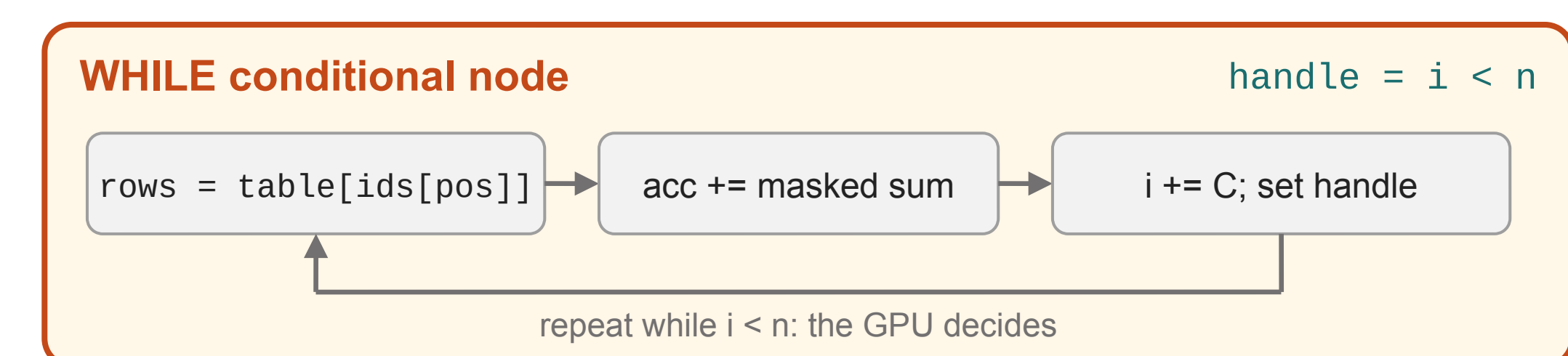
02 Runtime-sized embedding bags

`torch.while_loop()` → WHILE conditional node

```
def embedding_bag(table, ids, n):
    C = 4
    i = torch.zeros(), dtype=torch.int64, device=ids.device)
    acc = table.new_zeros(table.shape[1])

    def body(i, acc):
        pos = i + torch.arange(C, device=ids.device)
        rows = table[ids[pos.clamp_max(len(ids) - 1)]]
        return i + C, acc + (rows * (pos < n)[: , None]).sum(0)

    _, acc = torch.while_loop(
        lambda i, acc: i < n, body, (i, acc))
    return acc
```



`n` changes per replay; `acc` keeps its shape and address.

01 Skip non-finite optimizer steps

`torch.cond()` → IF/ELSE conditional node

```
def safe_sgd(param, grad, lr):
    finite = torch.isfinite(grad).all()
    return torch.cond(
        finite,
        lambda p, g, rate: p - rate * g,
        lambda p, g, rate: p.clone(),
        (param, grad, lr),
    )
```

Capture once, then replay:

```
step = torch.compile(safe_sgd, backend="cudagraphs")
updated = step(param, grad, lr) # replays one graph
```

The predicate never leaves the GPU: no `.item()`, no CPU sync.

The same graph takes or skips the update on every replay.

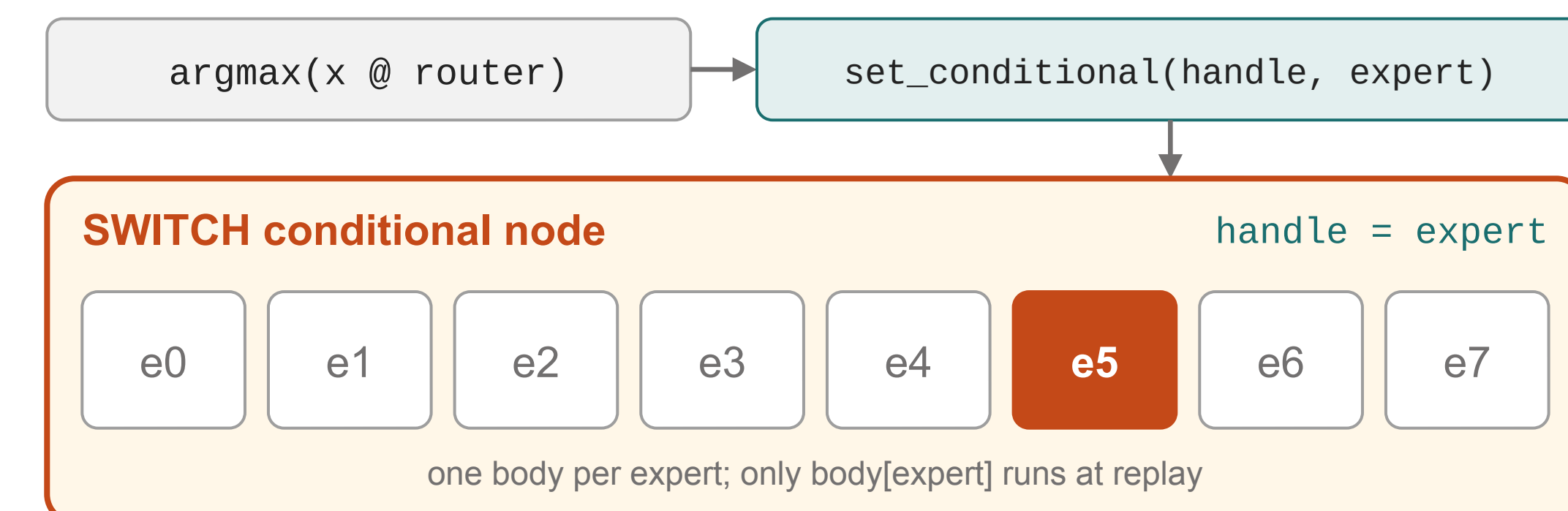
Before: a Python if on this flag forces a CPU sync and breaks capture.

03 Top-1 MoE: run only the routed expert

`torch.switch()` → SWITCH conditional node

```
from torch._higher_order_ops import switch

def moe_decode(x, router, w_up, w_down):
    expert = (x @ router).argmax()
    experts = [
        lambda x, e=e: torch.relu(x @ w_up[e]) @ w_down[e]
        for e in range(len(w_up))
    ]
    return switch(expert, experts, (x,))
```

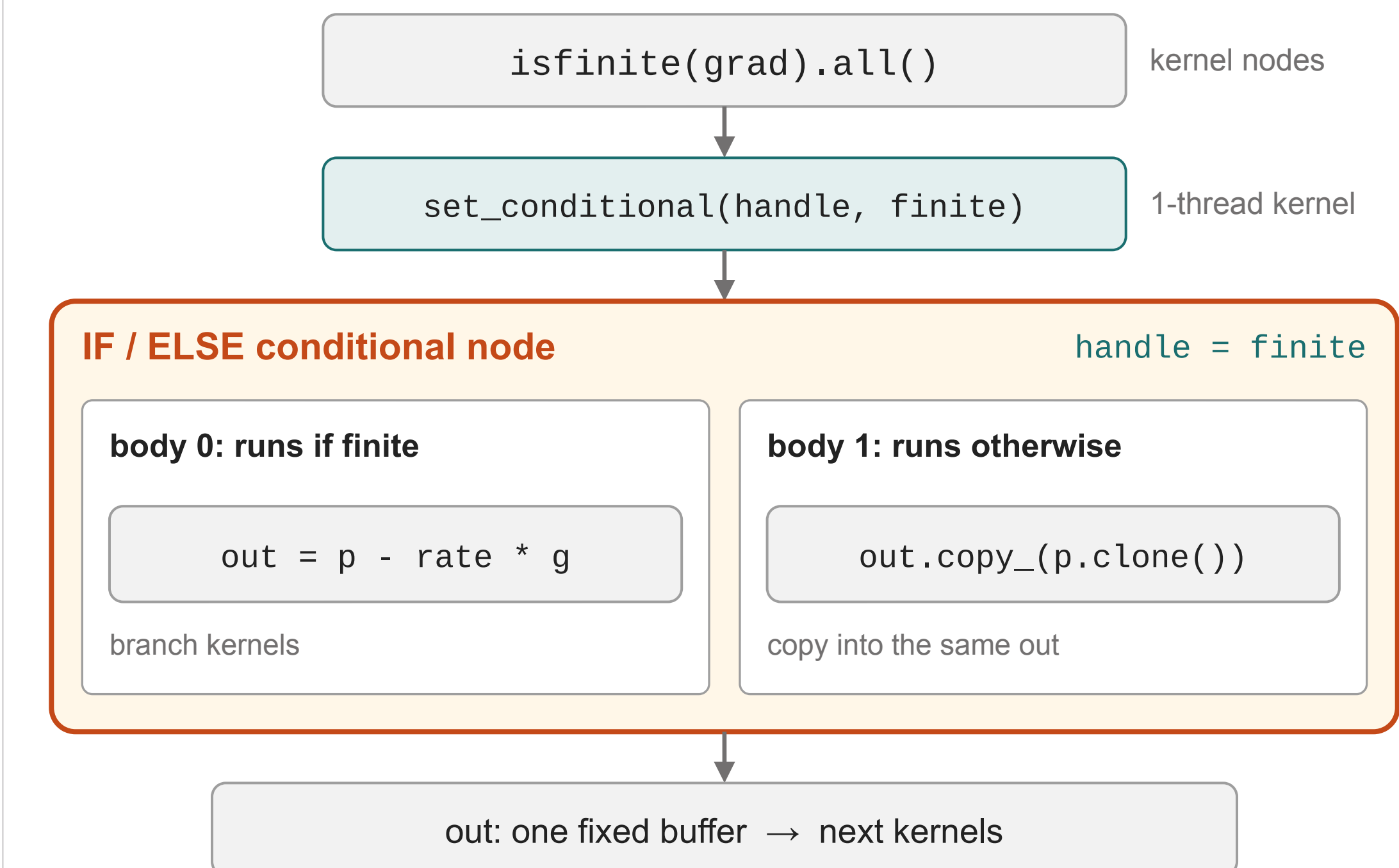


Only the routed expert's kernels run and only its weights are read.

Capture needs #189461; today the example runs eagerly / `aot_eager`.

The captured CUDA graph

What `safe_sgd` becomes: one graph, GPU-side branch



The contract

What keeps a region inside one CUDA graph

- **Static output shapes.** Every branch and loop iteration returns tensors of the same shape, dtype, and device.
- **Decisions stay on the GPU.** Predicates, loop guards, and switch indices are CUDA tensors; replay never calls `.item()`.
- **Bounded dynamism.** A runtime size becomes a GPU scalar plus a fixed-capacity buffer, as in the chunked `while_loop`.
- **One line to adopt.** `torch.compile(fn, backend="cudagraphs")` captures once and replays; no manual capture code.
- **Scales out (prototype).** EP=8 all-to-all MoE inside `torch.while_loop` on 8 H100s with zero dropped tokens.

Work Before General Use

Prototype; these upstream items are still open:

#186056 input mutation in control flow (in-place optimizer step)

#189462 a single IF/ELSE conditional node (CUDA 12.8)

#189461 `torch.switch()` to a SWITCH conditional node (CUDA 12.8)

#192098 reuse memory blocks across conditional streams

#191681 NCCL collectives inside conditional bodies

Not yet: conditional nodes generated by Inductor.